CAMPUS CAFE

CAFE MANAGEMENT SYSTEM

Student: RISHI GIRI GOSWAMI
Registration Number: 26BAI10756
Course: Python Essentials
Academic Year: 2026
Submission: September 2026

1. Introduction

Campus Cafe is a command-line based Cafe Management System developed in Python. The project was created as part of the Python Essentials evaluated course project. It focuses on applying basic Python programming concepts to a practical situation that is easy to understand and use.

2. Problem Statement

A small cafe needs a simple way to take customer orders and calculate bills accurately. Manual calculations can take time and may lead to mistakes. This project provides a basic terminal-based solution where customers can view a menu, place an order, select quantities, and receive a calculated bill.

3. Objectives

The main objectives are to create a practical Python application, understand how dictionaries can store menu data, use functions to divide a program into logical parts, validate user input, calculate discounts and GST, and create a friendly command-line interaction.

4. Scope of the Project

The current project is intended for a small cafe environment. It handles one customer order at a time. It supports dine-in and takeaway orders, menu selection, quantities, billing, discount calculation, GST calculation, and payment method selection. It is intentionally kept command-line based so that it can run easily in a terminal without a graphical interface or external service.

5. Features

The system accepts a customer name, provides dine-in or takeaway selection, asks for a table number for dine-in orders, displays the menu, accepts multiple food items, validates item names and quantities, calculates the subtotal, applies a 10% discount when the subtotal is at least Rs. 500, adds 5% GST after the discount, displays a final bill, and records the selected payment method.

6. Technologies Used

The project uses Python 3. It uses standard language features including dictionaries, functions, while loops, for loops, if-else conditions, exception handling with try-except, string formatting, arithmetic operations, and terminal input/output. No third-party Python packages are required.

7. Menu and Billing Logic

The menu is stored in a Python dictionary where each food item is associated with its price. When an item is ordered, its quantity is multiplied by the menu price. All item amounts are added to calculate the subtotal. If the subtotal is Rs. 500 or more, a 10% discount is applied. GST is then calculated at 5% on the discounted amount. The final amount is the discounted amount plus GST.

8. Program Flow

The program starts with a welcome message and asks for the customer's name. It then asks whether the order is dine-in or takeaway. For dine-in orders, a table number is collected. The menu is displayed and the customer can continue adding items until they choose to stop. The bill is calculated and displayed. Finally, the customer selects Cash, UPI, or Card as the payment method and receives a closing message.

9. Python Concepts Demonstrated

The project demonstrates dictionaries through the menu and order storage; functions through separate operations such as showing the menu, taking an order, calculating the bill, and choosing payment; loops for repeated ordering; conditions for validation and discount calculation; exception handling for invalid numeric input; and formatted strings for readable receipts.

10. Input Validation

The program checks whether the selected service type is valid. It checks whether a menu item exists before adding it. Quantity input is handled using try-except so that text does not crash the program. Quantities less than one are rejected. Payment selection is also checked against the available options.

11. Sample Output

A typical interaction starts with a welcome message, followed by customer details and menu selection. After the customer finishes ordering, the system prints a receipt containing the customer name, order type, individual item amounts, subtotal, discount, GST, and final amount. The program then records the payment method and ends with a friendly message.

12. Testing

The program was designed to handle normal orders as well as common incorrect inputs. Test cases include a valid single-item order, multiple items, repeated ordering of the same item, an invalid menu item, a non-numeric quantity, a zero or negative quantity, a bill below Rs. 500, a bill of Rs. 500 or more, and invalid payment input. The expected behavior is to continue running and guide the user instead of terminating unexpectedly.

13. Limitations

The current version does not store information permanently. Once the program closes, previous orders are not saved. There is no database, staff login, inventory tracking, online payment connection, or graphical user interface. These limitations keep the project simple and suitable for demonstrating core Python concepts.

14. Future Enhancements

The system could be expanded with a database, staff login, inventory management, order numbers, sales reports, persistent order history, receipt files, a graphical user interface, and an administrator panel for changing menu items and prices.

15. Conclusion

Campus Cafe demonstrates how basic Python programming can be used to solve a practical problem. The project combines input handling, data storage using dictionaries, functions, loops, conditions, validation, and calculations into one working application. The command-line design makes the project easy to run and understand while leaving clear possibilities for future development.

16. Project Files

File	Purpose
cafe_management.py	Main Python application
README.md	Setup, execution, and project information

requirements.txt	Documents that no external packages are required

17. Declaration

I understand that the submitted project should represent my own work and that I am responsible for reviewing, understanding, testing, and making any necessary changes to the project before submission.